NavCore

AUTONOMOUS NAVIGATION · OBSTACLE AVOIDANCE

NavCore

An autonomous differential-drive robot: technical documentation



Autonomous navigation · Obstacle avoidance · Wi-Fi telemetry

1. Introduction

1.1 What the robot does

NavCore is a small two-wheeled robot that drives from a start point A to a destination B along a straight, imaginary line. There is no tape on the floor and no camera. The robot works out where it is from its own wheel movement and from a gyroscope, and it notices obstacles with three laser distance sensors. When something blocks the way, it stops, decides which side offers more room, drives around the obstacle and then returns to the original line before continuing towards B.

Everything can be watched and controlled from a phone or laptop. The robot runs its own small web server, so opening its address in a browser shows a live map with the robot, its path, what the sensors see and every decision it takes. The same page contains an offline simulator that runs the identical avoidance logic, which made it possible to try ideas without risking the hardware.

1.2 System at a glance

Table 1. *Main subsystems.*

Part	Implementation
Controller	ESP32-WROOM-32, dual core: motion control on core 1, web server on core 0
Drive	Two DC gear motors, TB6612FNG dual H-bridge, one encoder per wheel
Sensing	Three VL53L0X time-of-flight laser sensors (front, left, right), MPU6050 gyroscope
Localisation	Dead reckoning: encoder distance combined with gyroscope heading
Obstacle handling	Reactive avoidance: detect, choose side, arc out, pass, arc back, realign
User interface	Browser application with live map, telemetry, event log, calibration and simulator
Route	Straight line A→B, 100 to 600 cm, up to 10 avoidances per run

2. Hardware

2.1 Platform and components

The chassis is a compact hand-built frame about 24 cm long and 16 cm wide (with wheels). All three distance sensors are mounted at the front edge, 13.5 cm ahead of the wheel axle: one looks straight ahead, the other two look sideways. This distance matters later, because every measurement is converted to the position of the axle, which is the point the robot actually tracks.

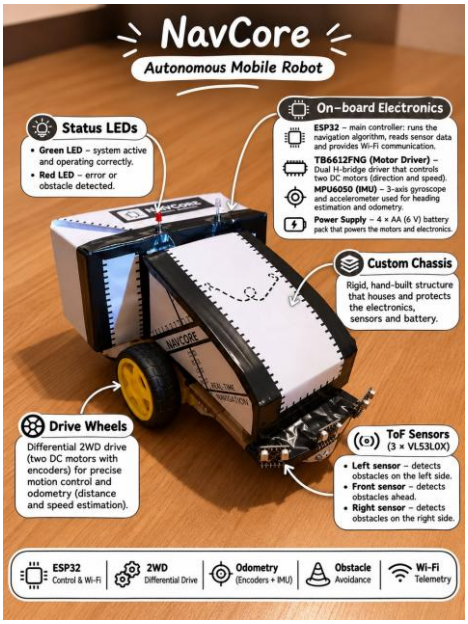


Figure 1. *The robot and its on-board components.*

Table 2. *Components and their role.*

Component	Role in the system
ESP32-WROOM-32	Runs navigation, motor control, sensor processing and the Wi-Fi web server
TB6612FNG	Sets direction and speed (PWM, 20 kHz) of each motor; supports active braking
Wheel encoders	Produce pulses as the wheels turn; counting them gives the distance travelled
MPU6050	Measures how fast the robot rotates around the vertical axis (Z gyroscope)
3 × VL53L0X	Measure distances in front and on both sides; readings beyond about 1.2 m count as free space
Status LEDs	Green: way clear. Red: obstacle ahead, avoidance in progress or safe stop

2.2 Wiring and pin assignment

All sensors share a single I²C bus (two wires: data and clock). The three VL53L0X sensors leave the factory with the same address, so each one has its own XSHUT line. At start-up the ESP32 keeps all three switched off, then wakes them one at a time and gives each a new address (0x30, 0x31, 0x32). The gyroscope keeps its default address 0x68.

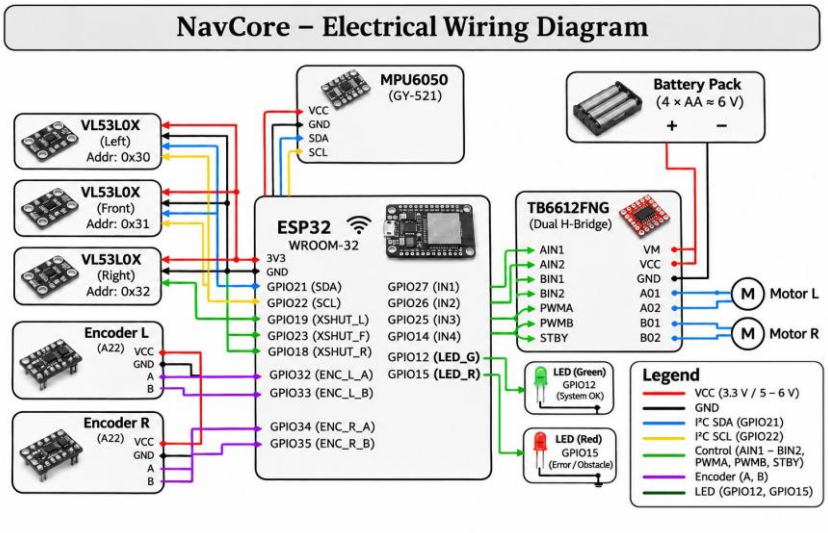


Figure 2. *Electrical wiring diagram.*

Table 3. *Pin assignment.*

Function	ESP32 pins
Right motor (TB6612 channel A)	AIN1 GPIO27 · AIN2 GPIO14 · PWMA GPIO13
Left motor (TB6612 channel B)	BIN1 GPIO25 · BIN2 GPIO26 · PWMB GPIO4 · STBY tied high
Encoders	left GPIO32 · right GPIO33 (interrupt on rising edge)
I²C bus	SDA GPIO21 · SCL GPIO22
VL53L0X XSHUT	left GPIO19 · front GPIO23 · right GPIO18
Status LEDs	red GPIO15 · green GPIO12, each through a 220–330 Ω resistor

2.3 Power supply

A pack of four AA cells (about 6 V) feeds two rails. The motors take their current directly from the battery through the TB6612FNG, while an LM2596S step-down converter produces the 5 V rail for the ESP32, the sensors and the LEDs. All grounds are joined. Motors draw short, large current peaks when they start, reverse or brake, and those peaks can pull the supply down for a few milliseconds. That is enough to reset a distance sensor, so two 470 µF electrolytic capacitors are placed as close as possible to where the current is needed: one across the driver supply (VM–GND) and one across the ESP32 input (VIN–GND).

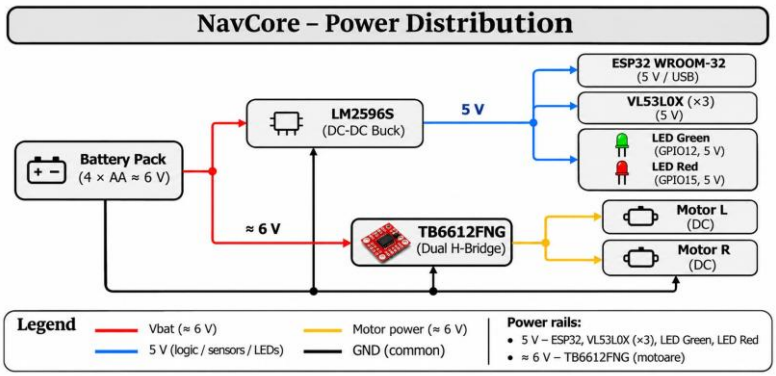


Figure 3. *Power distribution.*

3. Software structure

3.1 Two cores, two jobs

The ESP32 has two processor cores and the firmware uses both. Core 1 runs the control loop: it reads the sensors, updates the position estimate, drives the motors and takes every avoidance decision. Core 0 runs the HTTP server that answers the browser. The two sides only exchange a handful of shared variables (state, position, distances, a ring of events and a ring of trail points), so a slow Wi-Fi connection can never delay a braking decision.

3.2 Reading the sensors

Each VL53L0X measurement takes 20 ms. While driving straight only the front sensor is read, which keeps the reaction time short; the side sensors are read when the robot has to choose a side or follow the flank of an obstacle. A reading of 8190 means "nothing in range", while 65535 means the sensor did not answer at all. The firmware treats the second case as a fault, not as free space.

Two mechanisms keep the sensors alive. If any sensor stops answering, the robot brakes, frees the I²C bus (up to nine clock pulses followed by a STOP condition, the standard recovery for a device holding the data line low), re-runs the XSHUT start-up sequence and continues. The bus runs at 100 kHz with a 50 ms timeout, so a stuck transfer cannot freeze the loop. After a reboot the firmware also reports the reason, and a brown-out (supply too low) is shown in the application log.

4. Localisation: how the robot knows where it is

4.1 The coordinate frame

Everything is expressed relative to the route. The origin is the wheel axle at the start point A. The x axis points along the line towards B and the y axis measures the sideways offset from that line. The robot's heading θ is the angle between its forward direction and the route. When the robot is perfectly on track, y = 0 and θ = 0.

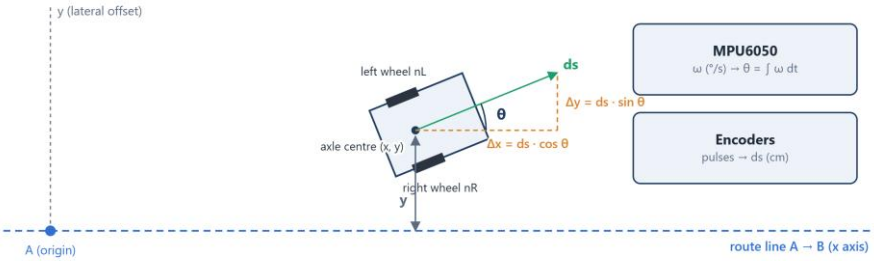


Figure 4. *Route frame: a small forward step ds splits into a part along the route and a part across it.*

4.2 Distance from the wheel encoders

Each wheel has an encoder that sends a pulse every time the wheel turns by a small fixed angle. Counting pulses tells how far each wheel has rolled. The distance travelled by the centre of the axle is the average of the two wheels:

$$\mathrm{ds} = \Delta((\mathrm{nL} + \mathrm{nR}) \,/\, 2) \,/\, \mathrm{PULSES_PER_CM}$$

Here nL and nR are the pulse counts of the left and right wheel and PULSES_PER_CM is the calibration constant (1.8768 pulses per centimetre on this robot, see Section 4.5). The encoders have a single channel, so they cannot tell forward from backward. The firmware knows when it is reversing and flips the sign of ds itself. During a turn on the spot both wheels roll but the axle centre does not move, so those pulses are ignored. Electrical noise from the motors can create false pulses; a pulse is accepted only if at least 7 ms have passed since the previous one and the pin is still high.

4.3 Heading from the gyroscope

The MPU6050 does not measure an angle directly. It measures angular velocity, in degrees per second. Adding up these small rotations over time (integration) gives the heading:

$$\omega = \mathrm{raw_z} \,/\, 65.5 - \mathrm{offset} \qquad \theta(\mathrm{k}) = \theta(\mathrm{k}-1) + \omega \cdot \Delta\mathrm{t} \cdot \mathrm{GYRO_SCALE}$$

The gyroscope is configured for ±500 °/s, which corresponds to 65.5 raw units per °/s. Even at rest a gyroscope reports a small non-zero rate (the offset). If it were not removed, the heading would drift steadily. The robot therefore averages 1000 samples (about two seconds) while standing still at every start. GYRO_SCALE corrects the small scale error of this particular chip and was measured with a full 360° turn (0.9989). Integration continues during braking pauses as well, because the robot keeps rotating for a fraction of a second after the motors stop.

4.4 Updating the position

Combining the two measurements, every new piece of distance is split into its component along the route and its component across it:

$$\mathrm{x} \leftarrow \mathrm{x} + \mathrm{ds} \cdot \cos\theta \qquad \mathrm{y} \leftarrow \mathrm{y} + \mathrm{ds} \cdot \sin\theta$$

A short example makes this concrete. Suppose the robot is heading 10° away from the line and the encoders report 2 cm of travel. The robot advances 2 · cos 10° = 1.97 cm along the route and drifts 2 · sin 10° = 0.35 cm sideways. Repeated hundreds of times per second, these steps add up to the full path drawn on the live map. This method is called dead reckoning. It is accurate over short distances, but every small error stays in the sum, which is why calibration matters.

4.5 Calibration

Two constants are calibrated from the application and stored in the ESP32 flash memory, so they survive a power cycle. For distance, the robot is asked to drive 200 cm and the real distance is measured with a tape. The ratio corrects PULSES_PER_CM. For rotation, the robot turns 360° on the spot and the difference between the gyroscope angle and the real angle gives GYRO_SCALE. With the default constant the robot drove 207–209 cm instead of 200 cm. After calibration a 200 cm command produced 200 cm.

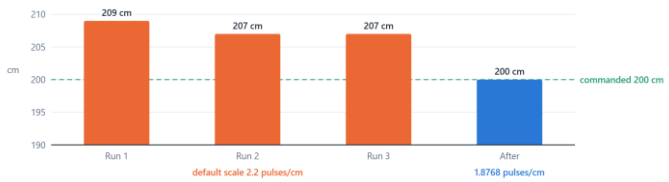


Figure 5. *Distance travelled for a 200 cm command, before and after calibration.*

5. Motion control

5.1 Driving straight and holding the line

Two identical motors never turn at exactly the same speed, so a robot given equal power drifts. NavCore corrects this continuously with a proportional controller: the difference between the wanted heading and the measured heading is multiplied by a gain and added to one wheel and subtracted from the other.

$$\text{error} = \theta - \theta_\text{goal} \qquad \text{correction} = \text{clamp}(2.0 \cdot \text{error}, \pm 20)$$

$$\text{PWM_left} = 130 + 7 - \text{correction} \qquad \text{PWM_right} = 130 - 7 + \text{correction}$$

The base power is 130 out of 255 (about 21 cm/s) and the fixed trim of ±7 compensates the natural difference between the two motors. The goal heading is not simply "parallel to the route". The robot aims at a point on the line 35 cm ahead of it, so any sideways offset left after an avoidance is removed gradually while driving:

$$\theta_\text{goal} = -\text{clamp}(\text{atan2}(y, 35\ \text{cm}), \pm 20°)$$

For example, 10 cm off the line the robot steers atan2(10, 35) ≈ 16° back towards it. At 2 cm it steers only about 3°, so it settles onto the line smoothly instead of oscillating across it.

5.2 Turning on the spot

A pivot turn spins one wheel forward and the other backward, so the robot rotates around the middle of its axle. The difficulty is stopping at the right angle: a moving robot keeps rotating for several degrees after the brake is applied. NavCore handles this in two stages.

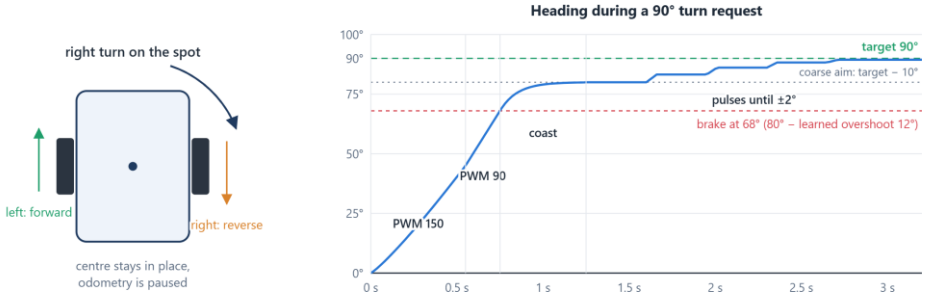


Figure 6. *Pivot turn (left) and the heading during a 90° request (right).*

Coarse stage. For errors above 25° the robot aims 10° short of the target. It turns at PWM 150, slows to PWM 90 after 45°, and applies an active brake early, at that angle minus the expected overshoot. It then keeps reading the gyroscope for 1.5 s to see how far it actually coasted. That measured coast updates the overshoot estimate for the next turn, which makes the turns more accurate over time:

$$\text{overshoot} \leftarrow 0.6 \cdot \text{overshoot} + 0.4 \cdot \text{clamp}(\text{coast}, 2°, 22°)$$

Fine stage. The remaining error is removed with short pulses. Each pulse lasts 2 ms per degree of error (between 25 and 100 ms), followed by a 300 ms pause to measure the result. If a pulse moved the robot by less than 1°, the next one gets 15 ms more, which overcomes static friction. The turn is finished when the error is within ±2°.

5.3 Driving on arcs

Leaving the line and coming back to it are done on smooth arcs rather than with sharp pivots, because arcs keep the robot moving and lose less position accuracy. During an arc the same heading controller is used with a wider correction range (±50). The tricky part is knowing when to start straightening out: the robot keeps gaining sideways distance while it turns back to parallel. This extra distance was measured from many logged runs and is predicted with:

$$y + K \cdot (1 - \cos\theta) \geq y_\text{goal} \qquad K = 20.5\ \text{cm (outward)}, 17.5\ \text{cm (return)}$$

As soon as this prediction reaches the target, the robot starts turning back, and it ends up parallel to the route almost exactly at the planned offset.

6. Obstacle avoidance

6.1 The run as a state machine

A run is organised as a sequence of states, and the current state is shown live in the application. After Start the robot measures the gyroscope offset (CALIB), then follows the line (DRIVE). An obstacle sends it through CHOOSE, OUT, RET and ALIGN and back to DRIVE. Two exits end the run: ARRIVED when the route length is covered, and FAILED when the robot decides it cannot pass safely and stops. BACK is used when something new appears in the middle of a manoeuvre.

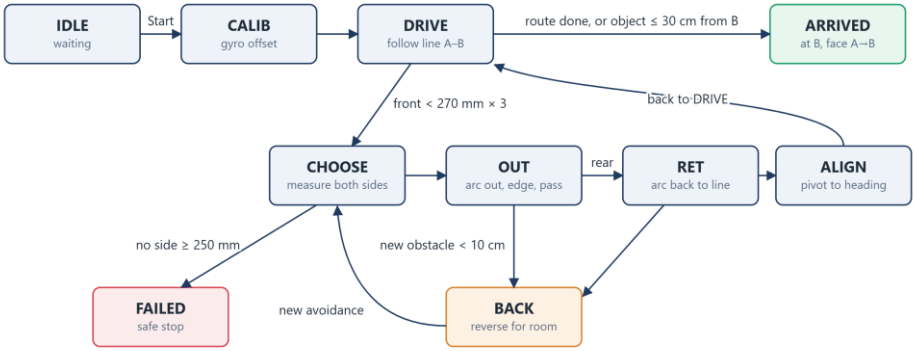


Figure 7. *States of a run, as reported by the robot.*

6.2 Detecting an obstacle

A single short reading is not trusted, since reflections and noise do occur. An obstacle is confirmed after **three consecutive front readings below 270 mm**. There is one special case: when an object gets very close, the VL53L0X may report "nothing in range". An invalid reading that comes right after a valid one below 300 mm is therefore counted as an obstacle, not as free space. If the robot ends up closer than 220 mm, it first reverses to about 25 cm so the outward arc has room. An object within 30 cm of B, or in the last 40 cm of the route, is treated as the destination (for example a wall) and ends the run as an arrival.

6.3 Choosing a side

With the robot stopped, the left and right sensors are read three times and averaged. A side is usable only if it offers at least 250 mm; if both do, the robot picks the side with more space. If neither does, the run stops safely, because driving into a gap that is too narrow is worse than not moving.

6.4 Finding the edge and setting the target

The robot does not know how wide the obstacle is, so it measures it on the way. First, the front reading at the stop gives the position of the obstacle face. While the robot arcs outward, the front beam sweeps across the obstacle. Each reading that still lands on the face gives a point on it, and the moment the beam slips past the corner gives the edge. Because the beam is a cone rather than a thin line, the edge is pulled in by a 6° correction.

$$\text{x_face} = 13.5 + \text{d_front} \qquad \text{y_edge} = \text{y_hit} - \text{d} \cdot \tan 6°$$

$$\text{y_goal} = \text{clamp}(\text{y_edge} + 8 + 4,\ 12,\ 55)\ \text{cm}$$

The 8 cm is half the robot width and the 4 cm is the clearance between the robot and the box. If y_goal is larger than the free space measured on that side (minus a 6 cm margin), that side is abandoned and the other one is tried.

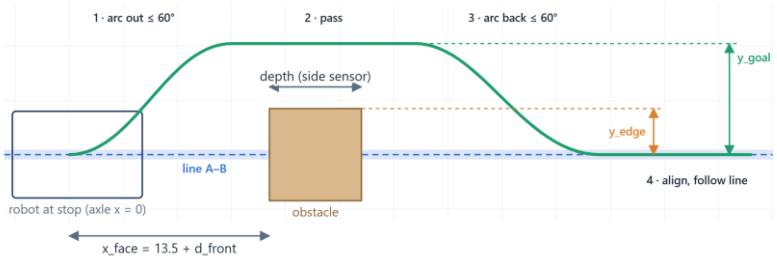


Figure 8. *Geometry of one avoidance and the quantities the robot measures.*

6.5 Passing the obstacle and finding its rear

While the robot drives parallel to the route, the side sensor facing the obstacle keeps track of it and records the farthest point it touches. The rear of the obstacle is confirmed only after **six consecutive "clear" readings**, and a measured depth below 12 cm is treated as a momentary loss of the target rather than a real corner. Both rules exist for the same reason: if the robot decides too early that it has passed, it turns back while the box is still beside it. If the side sensor never sees the obstacle at all, the robot assumes a depth of 20 cm (adjustable from the app) and never passes more than 90 cm beyond the face.

6.6 Returning to the route

The distance back to the line is taken from the odometry (the current y). The robot follows a return arc of at most 60°, straightens when the angle to the route drops below 4°, pivots to within ±2° of the route heading and resumes DRIVE. The look-ahead steering of Section 5.1 removes whatever offset is left.

6.7 Avoidance scenarios

Four scenarios cover the behaviours above. They are also the cases shown in the demonstration video.

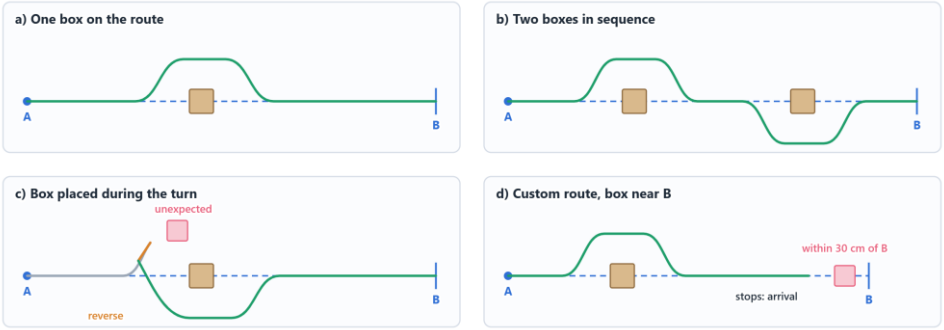


Figure 9. *The four demonstrated scenarios (schematic).*

One box on the route. The basic case: detect, choose side, arc out, pass, arc back, realign and continue to B.

Two boxes in sequence. After the first avoidance the robot is back on the line and in DRIVE, so the second box is handled as a completely new obstacle. Each box gets its own side decision, so the robot may pass them on opposite sides.

Box placed during the turn. If something appears closer than 10 cm while the robot is arcing, it cannot continue. It brakes, reverses to about 25 cm, aligns with the route and starts a fresh avoidance from that position (state BACK). If the same side is blocked again, it backs off to 35 cm and tries the other side.

Custom route with a box near B. Route length and obstacles are set in the app. An object that appears within 30 cm of B is not avoided: the robot interprets it as the end of the route, stops and reports the arrival. Further from B, the same object would be avoided normally.

6.8 Safety rules

Table 4. *Built-in safety behaviour.*

Situation	Robot behaviour
No side has 250 mm free	Safe stop (FAILED), nothing is forced

Situation	Robot behaviour
Object < 100 mm during OUT or RET	Brake, reverse, start a new avoidance
Sensor stops answering	Brake, recover the I²C bus, re-initialise, continue
More than 10 avoidances	Run is stopped
Stop command from the app	Immediate stop, from any state

7. Web application

7.1 What the user sees

The application is a single HTML page. It is compressed with gzip and stored inside the firmware, so the robot needs no SD card and no internet connection. The upper part shows status cards (state, position, angle, the three distances, progress, number of avoidances, connection quality). Below them is the map with the planned line, the boxes placed by the user, the path actually driven and the sensor cones. Obstacles the robot measured are outlined on the map; one that was not placed by the user is drawn in pink and labelled as unexpected.



Figure 10. *The application connected to the robot after a run with two avoidances.*

Table 5. *Application features.*

Feature	Purpose
Control	Start, stop and reset; route length from a field, a slider or by dragging B on the map
Map editor	Place, move, resize and delete boxes; save custom scenarios
Event log	Every decision with its time and values (distances, side, edge, rear, final offset)
Export	The complete run (trail, obstacles, events) saved as a JSON file for analysis
Calibration	Distance and rotation calibration, stored on the robot
Simulator	The same avoidance logic running in the browser, without the robot

7.2 Communication with the robot

The robot joins a Wi-Fi network (a phone hotspot during tests). If that fails within 30 seconds, it creates its own access point and serves the page at 192.168.4.1. All communication is plain HTTP and is always started by the browser.

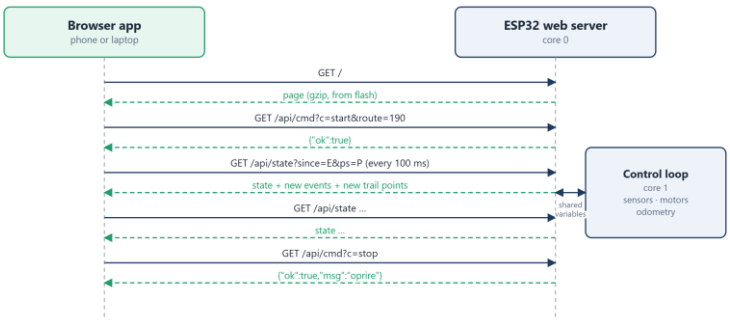


Figure 11. *How the application and the robot exchange data.*

Every 100 ms the page asks for /api/state and tells the robot which event and which trail point it received last. The robot answers with a short JSON message containing the current state and only what is new. The robot keeps the last 16 events and the last 512 trail points (one every centimetre or so) in ring buffers. When a few replies are lost, the next reply fills the gap, so the drawn path has no missing segments. A request that gets no answer within 1.2 s counts as failed, and after four failures in a row the page shows the robot as disconnected. Commands use /api/cmd.

Table 6. *HTTP interface.*

Request	Effect
GET /	Returns the application
GET /api/state?since=E&ps=P	State, position, distances, new events and new trail points
GET /api/cmd?c=start&route=L	Starts a run of L cm
GET /api/cmd?c=stop · c=reset	Stops the run · sets the position back to A
GET /api/cmd?c=calib&ppcm=…&gs=…	Stores new calibration constants

7.3 The simulator

Without a robot connected, the Start button runs the simulator. It models the robot's size, speed and sensor cones and uses the same thresholds and the same avoidance steps as the firmware. Because the map editor works for both, a scenario can be drawn once, tried in simulation and then run on the real robot with the same layout.

8. Tests and results

8.1 Test setup

All tests were done indoors on a smooth floor, with cardboard boxes as obstacles (17 × 17, 29 × 11, 33 × 17 and 11 × 5 cm) and routes between 190 and 420 cm. Every run was exported from the application as a JSON file containing the full trail, the measured obstacles and the event log. The values below come from those files. Positions are the robot's own estimates unless a tape measurement is mentioned.

8.2 Straight driving and turning

Before the avoidance logic was added, the robot drove a square path made of four 50 cm sides and three 90° right turns. The two encoders counted 88 to 90 pulses on every side, which shows that the straight-line controller keeps both wheels within about one pulse of each other.

Table 7. *Final heading of the 90° pivot turns (gyroscope).*

Turn	Requested	Measured	Error
2	90°	89.49°	−0.51°
3	90°	91.11°	+1.11°
4	90°	88.46°	−1.54°
Mean	90°	89.69°	range 2.65°

8.3 Measured avoidances

Table 8 lists avoidances taken from the event logs, each on a 17 × 17 cm box. In every row the lateral target equals the measured edge plus 12 cm, exactly as defined in Section 6.4, which confirms that the geometry is computed as designed. The last column is how far from the line the robot was when the avoidance ended.

Table 8. *Logged avoidances (values copied from the robot event log).*

#	Front at stop	y_edge	y_goal	Depth	Return (dist, angle)	Final offset
1	259 mm	5.1 cm	17.1 cm	14.7 cm	21.9 cm, 5.6°	0.4 cm
2	256 mm	12.6 cm	24.6 cm	—	26.7 cm, 2.8°	0.8 cm
3	249 mm	7.0 cm	19.0 cm	15.0 cm	23.1 cm, 5.1°	0.7 cm
4	264 mm	14.1 cm	26.1 cm	—	27.6 cm, 4.2°	0.7 cm
5	254 mm	5.5 cm	17.5 cm	14.2 cm	21.5 cm, 4.9°	0.7 cm
6	263 mm	4.3 cm	16.3 cm	—	21.1 cm, 3.3°	1.1 cm
7	250 mm	6.3 cm	18.3 cm	—	16.4 cm, 0.5°	1.9 cm
8	—	5.5 cm	17.5 cm	—	21.2 cm, −0.7°	1.1 cm
9	241 mm	8.5 cm	20.5 cm	—	23.1 cm, 1.8°	1.1 cm

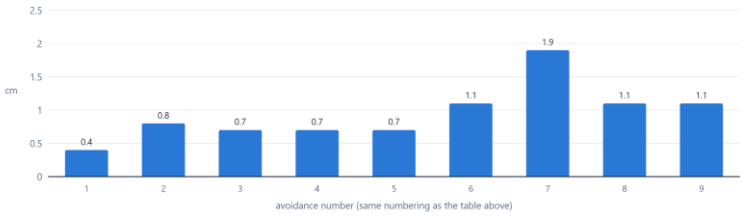


Figure 12. *Distance from the route line at the end of each avoidance (robot estimate).*

The robot finished every avoidance within 2 cm of the line by its own estimate, with a typical value below 1 cm. The measured depth of the 17 cm boxes came out at 14–15 cm; the difference is the width of the side sensor's beam, which reaches the rear corner slightly before the sensor itself does.

8.4 A complete run

Figure 13 shows the path recorded during a 190 cm run with one box on the line. The robot detected the box at 250 mm, found its edge at 6.3 cm, passed on the left with an 18.3 cm offset, returned to the line and stopped at B after 16.7 s, 0.9 cm from the line.

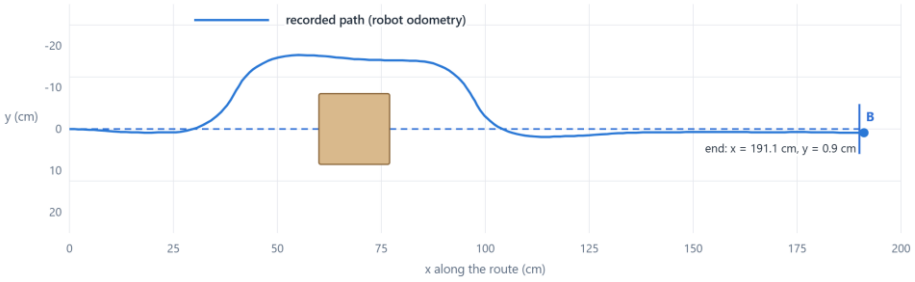


Figure 13. *Recorded path of a 190 cm run with one obstacle (17 × 17 cm box).*

8.5 Scenario results

Table 9. *Behaviour in the test scenarios.*

Scenario	Observed behaviour
One or two boxes, 190 cm	In a series of six consecutive runs all six reached B; in one the return to the line was incomplete and in one the robot passed very close to a box
Box inserted during a manoeuvre	Handled by reversing and restarting the avoidance; needs free space behind the robot
Box near the destination	Recognised as the end of the route; the robot stops and reports the arrival
Three boxes on 375–420 cm	The most demanding case; errors accumulate over several avoidances and reversals

Most failures during development had one of three causes: loose wiring or supply dips that reset a sensor, an edge or rear decision taken on a single unlucky reading, and boxes that touched the route only with a corner. The first was solved in hardware and by the automatic sensor recovery, the second by the stricter confirmation rules described in Chapter 6. The third is a physical limit of the front sensor: its cone is only about 25° wide, so at 30 cm it covers roughly ±6 cm, while the robot is 16 cm wide. A box placed so that it only clips the robot's outer edge may not be seen in time.

9. Conclusion

NavCore shows that reliable short-range navigation is possible with simple, inexpensive parts and no external reference. Encoders and a gyroscope, combined through dead reckoning and calibrated against a tape measure, keep the position error within a couple of centimetres over a typical run. Three single-point laser sensors are enough to detect an obstacle, measure its edge and depth on the move and plan a path around it. The browser application makes every step visible, which proved to be the most useful tool of the whole project: almost every improvement started from a logged run whose numbers explained what the robot had actually seen and decided.